

Práctica Django ORM

Introducción al shell de Django

Es una herramienta interactiva para ser usada en consola que permite trabajar directamente con los modelos y la base de datos de un proyecto específico de Django

- Permite ejecutar código Python
- Permite consultar y manipular la base de datos a través del ORM

Ingresar al shell

Desde la raíz de tu proyecto Django se ejecuta:

```
python manage.py shell
```

Importar y usar modelos

```
from app.models import UnModelo, OtroModelo

# Crear objetos
unObjeto = UnModelo(un_atributo="Atributo", otro_atributo=12)
unObjeto.save()

# Consultar objetos
objetos = UnModelo.objects.all()

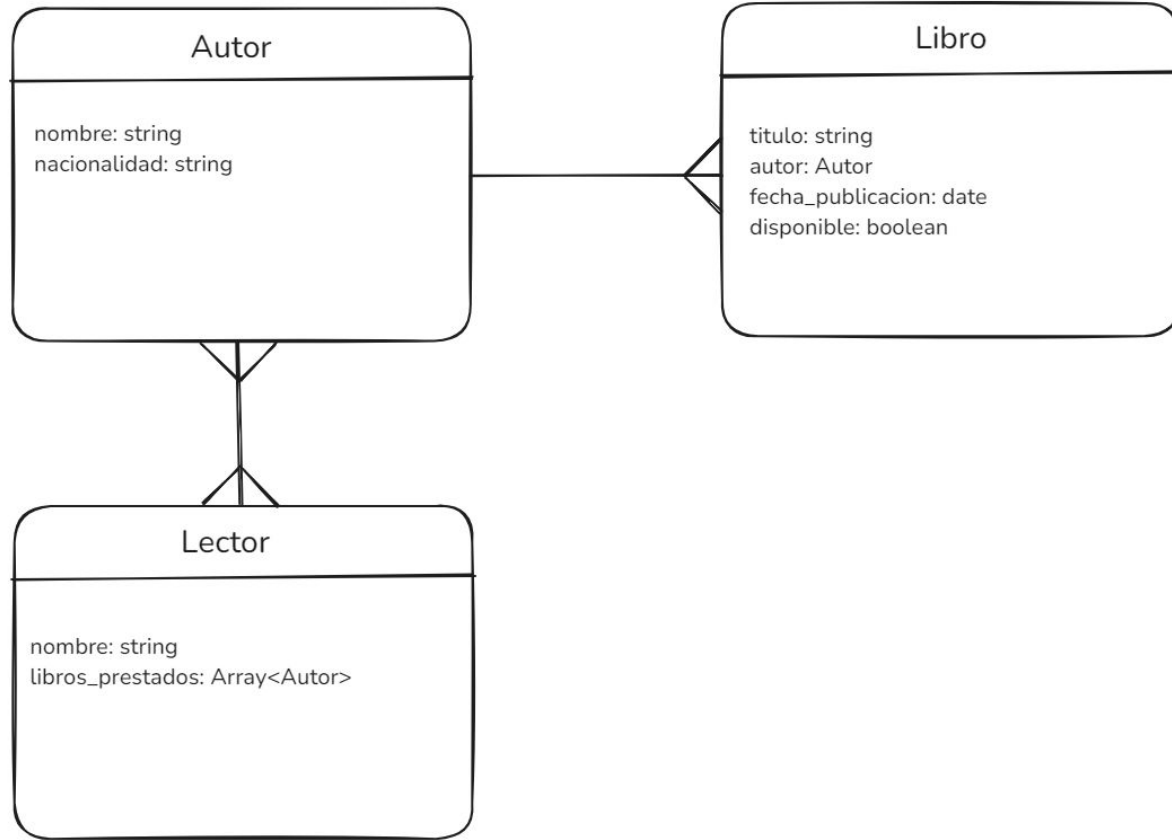
# Filtrar
objetos = UnModelo.objects.filter(otro_atributo=12)

# Ver todos
for objeto in objetos:
    print(objeto.un_atributo)
```

Actividad para la práctica

Basándonos en el siguiente Diagrama Entidad-Relación:

- Realizar un proyecto Django
- Crear una app en ese proyecto
- Realizar los modelos correspondientes



Crear y guardar objetos

1. Crear una instancia de Autor y guardarla en la base de datos.
2. Crear dos libros distintos vinculados al autor anterior y guardarlos en la base. Uno de ellos debe tener fecha 2020.
3. Crear un lector y guardarlo. Luego agregarle uno de los libros prestados.

Consultas

1. Obtener todos los libros disponibles.
2. Buscar todos los libros publicados en el año 2020.
3. Obtener el autor de un libro específico.

Filtros

1. Obtener todos los libros no disponibles.
2. Buscar todos los lectores que tienen prestado un libro cuyo título contiene la palabra "Python".
3. Obtener todos los libros cuyo autor sea de nacionalidad "Argentina".

Modificaciones

1. Cambiar el estado de disponibilidad de un libro a False.
2. Agregar un segundo libro a un lector ya existente.
3. Cambiar el nombre de un autor ya creado y guardar los cambios.

Ordenamiento

1. Obtener todos los libros ordenados por título en orden ascendente.
2. Obtener todos los libros ordenados por fecha de publicación, del más reciente al más antiguo.
3. Obtener todos los lectores ordenados alfabéticamente por nombre.
4. Obtener todos los libros ordenados por nacionalidad del autor (usando la relación con autor).

Borrado

1. Borrar un lector de la base de datos.
2. Borrar todos los libros que no están disponibles.

Managers y métodos

Un Manager es la interfaz a través de la cual se proporcionan las operaciones de consulta de la base de datos a los modelos de Django.

- **Por defecto:** Cada modelo en Django tiene al menos un Manager. Si no defines uno, Django añade automáticamente el Manager predeterminado llamado `objects`.
- **Patrón de Diseño:** Si vienen de Java o C#, piensen en el Manager como una implementación del Patrón Repository o DAO (Data Access Object).

La sintaxis que ya conocen: Cuando escriben `Libro.objects.all()` o `UnModelo.objects.filter(...)`, están usando los métodos `.all()` y `.filter()` proporcionados por el Manager `objects`

Managers y métodos

A medida que tu aplicación crece, empezarás a repetir los mismos filtros por todo el código (ejemplo: buscar solo los libros que no están dados de baja). Los Managers personalizados aplican el principio DRY (Don't Repeat Yourself).

Casos de uso principales:

- **Añadir métodos extra a nivel de tabla:** Operaciones que actúan sobre el conjunto de datos (ej: `Libro.objects.obtener_promedio_paginas()`).
- **Modificar el QuerySet inicial:** Cambiar lo que devuelve `.all()`. Por ejemplo, en lugar de devolver todos los libros de la base de datos, crear un manager `Libro.disponibles.all()` que oculte por defecto los libros prestados.

Managers y métodos

Tipo de Método	¿Dónde se define?	¿Sobre qué actúa? (Ámbito)	Ejemplo de Uso
Método de modelo	En la clase del Modelo	Nivel Fila (Instancia): Actúa sobre un único registro específico.	<code>un_lector.cantidad_libros_prestados()</code>
Manager / QuerySet	En una clase Manager	Nivel Tabla (Colección): Actúa sobre todo el conjunto de datos o devuelve múltiples filas.	<code>Libro.disponibles.all()</code>

Managers y métodos

1. Crear un método en el modelo Lector que devuelva la cantidad total de libros que tiene prestados.
2. Crear un Manager personalizado para Libro que filtre solo los disponibles.
3. Usar el nuevo manager para listar los libros disponibles ordenados por título.